UNIVERSIDAD EDEC

INGENIRIA EN SIETMAS

PROGRAMACION ORIENTADA A OBJETOS

Estudiante: Paulo Cesar Ramirez Rodriguez
No. de Matrícula: 41212

Ciclo Escolar: C3. Septiembre – Diciembre 2026

Docente: Alejandro Carbajal

UNIDAD 2 Implementación de Patrones de Diseño y Principios SOLID

Monterrey, N.L 5 DE NOVIEMBRE 2026

1. Introducción

El proyecto Sistema de Gestión de Pedidos consiste en una aplicación de consola desarrollada en Java que permite registrar pedidos normales y urgentes, almacenar la información en una base de datos SQLite, consultar los pedidos registrados y notificar a los usuarios que se hayan suscrito al sistema.

El propósito del desarrollo es aplicar conceptos de programación orientada a objetos, patrones de diseño y principios SOLID mediante una solución modular. La aplicación utiliza Maven para administrar el proyecto y la dependencia JDBC necesaria para la comunicación con SQLite.

2. Objetivo general

Desarrollar un sistema de gestión de pedidos que permita crear y consultar pedidos de diferentes tipos, conservar la información en una base de datos y notificar automáticamente a los usuarios suscritos, aplicando los patrones Factory, Singleton y Observer, así como los principios SOLID.

3. Objetivos específicos

Implementar la creación de pedidos normales y urgentes mediante el patrón Factory.

Centralizar la conexión con SQLite mediante el patrón Singleton.

Notificar a los usuarios suscritos cuando se registra un nuevo pedido mediante Observer.

Separar responsabilidades mediante clases, interfaces y servicios.

Aplicar los principios SOLID para favorecer la organización, extensibilidad y mantenimiento del código.

Comprobar el funcionamiento mediante pruebas de registro y consulta de pedidos.

4. Herramientas y tecnologías

Para el desarrollo del proyecto se utilizaron las siguientes herramientas y tecnologías:

- Lenguaje de programación: Java 25.
- Entorno de desarrollo: Apache NetBeans.
- Gestor de dependencias y construcción: Maven.
- Base de datos: SQLite.
- Conector de base de datos: SQLite JDBC, versión 3.50.3.0.
- Interfaz: menú de consola.

5. Descripción general del sistema

Al iniciar, el programa obtiene la conexión a la base de datos y muestra un menú con cinco opciones: crear pedido normal, crear pedido urgente, registrar usuario, consultar pedidos y salir.

Al crear un pedido, el sistema solicita el nombre del producto y su precio. Después, la fábrica crea el objeto correspondiente, el repositorio lo almacena en SQLite y el servicio solicita al evento que notifique a los observadores registrados.

Los usuarios registrados como observadores reciben una notificación en la consola cada vez que se crea un pedido mientras permanezcan suscritos. La consulta de pedidos recupera los registros almacenados, incluyendo su identificador, producto, precio y tipo.

SistemaPedidos.java | pom.xml [SistemaPedidos] | Pedido.java | PedidoNormal.java | PedidoUrgente.java | Usuario.java | PedidoFactory.java | ConexionBD.java | Observador.java | Evento.java | UsuarioOb...

Source | History

```java
package com.sistemadepedidos;

import com.sistemadepedidos.modelos.Usuario;
import com.sistemadepedidos.observer.Evento;
import com.sistemadepedidos.observer.UsuarioObservador;
import com.sistemadepedidos.servicios.RepositorioPedidosSQLite;
import com.sistemadepedidos.servicios.ServicioPedidos;
import com.sistemadepedidos.singleton.ConexionBD;

import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.Statement;
import java.sql.SQLException;
import java.util.Scanner;

public class SistemaPedidos {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        Evento evento = new Evento();

        ServicioPedidos servicio = new ServicioPedidos(
            new RepositorioPedidosSQLite(), evento
        );

        try {
            ConexionBD.getInstancia().getConexion();

            int opcion;

            do {
                System.out.println("\n============================");
                System.out.println("     SISTEMA DE PEDIDOS");
                System.out.println("============================");
                System.out.println("1. Crear pedido normal");
                System.out.println("2. Crear pedido urgente");
                System.out.println("3. Registrar usuario");
                System.out.println("4. Consultar pedidos");
                System.out.println("5. Salir");
                System.out.print("Selecciona una opcion: ");

```

Output

Retriever Output | Run (SistemaPedidos)

```
  Tipo: Urgente

  ================================
        SISTEMA DE PEDIDOS
  ================================
  1. Crear pedido normal
  2. Crear pedido urgente
  3. Registrar usuario
  4. Consultar pedidos
  5. Salir
  Selecciona una opcion: 4

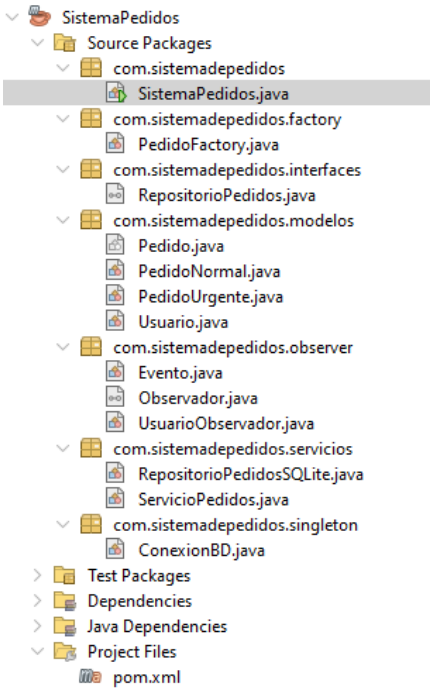
  ===== PEDIDOS REGISTRADOS =====
  ----------------------------
  ID: 1
  Producto: Pedido1
  Precio: $1800.0
  Tipo: Normal
  ----------------------------
  ID: 2
  Producto: Pedido2
  Precio: $1900.0
  Tipo: Urgente
  ----------------------------
  ID: 3
  Producto: Laptop Asus Gamer
  Precio: $18000.0
  Tipo: Normal
  ----------------------------
  ID: 4
  Producto: Mouse Gamer Asus
  Precio: $1400.0
  Tipo: Urgente
  ----------------------------
  ID: 5
  Producto: Laptop Asus
  Precio: $17000.0
  Tipo: Urgente
  ----------------------------
  ID: 6
  Producto: MiniLed Pantalla
  Precio: $5200.0
  Tipo: Urgente

  ================================
```

6. Organización del proyecto

El proyecto se organiza en paquetes para separar las responsabilidades de cada componente:

- com.sistemadepedidos: contiene la clase principal SistemaPedidos, encargada de mostrar el menú e interactuar con el usuario.
- com.sistemadepedidos.modelos: contiene las clases Pedido, PedidoNormal, PedidoUrgente y Usuario.
- com.sistemadepedidos.factory: contiene PedidoFactory, responsable de crear el tipo de pedido solicitado.
- com.sistemadepedidos.singleton: contiene ConexionBD, que administra la instancia de conexión a SQLite.
- com.sistemadepedidos.observer: contiene Observador, Evento y UsuarioObservador para administrar las suscripciones y notificaciones.
- com.sistemadepedidos.interfaces: contiene RepositorioPedidos, la abstracción para guardar pedidos.
- com.sistemadepedidos.servicios: contiene RepositorioPedidosSQLite y ServicioPedidos, que implementan el almacenamiento y coordinan el registro.



7. Patrones de diseño implementados

Patrón Factory

El patrón Factory se implementa mediante la clase PedidoFactory. Su método crearPedido recibe el tipo, el nombre del producto y el precio, y devuelve un objeto PedidoNormal o PedidoUrgente según la opción seleccionada. De esta manera, la lógica principal no necesita construir directamente las clases concretas.

La clase abstracta Pedido define los datos y el comportamiento común. PedidoNormal y PedidoUrgente heredan de ella y proporcionan su propio tipo mediante el método getTipo().

PedidoFactory.java

```java

package com.sistemadepedidos.factory;

import com.sistemadepedidos.modelos.Pedido;
import com.sistemadepedidos.modelos.PedidoNormal;
import com.sistemadepedidos.modelos.PedidoUrgente;

public class PedidoFactory {

    public static Pedido crearPedido(
            String tipo, String producto, double precio) {

        switch (tipo.toLowerCase()) {

            case "normal":
                return new PedidoNormal(producto, precio);

            case "urgente":
                return new PedidoUrgente(producto, precio);

            default:
                throw new IllegalArgumentException(
                    "Tipo de pedido no válido: " + tipo
                );
        }
    }
}


```

PedidoFactory.java | PedidoNormal.java

```java

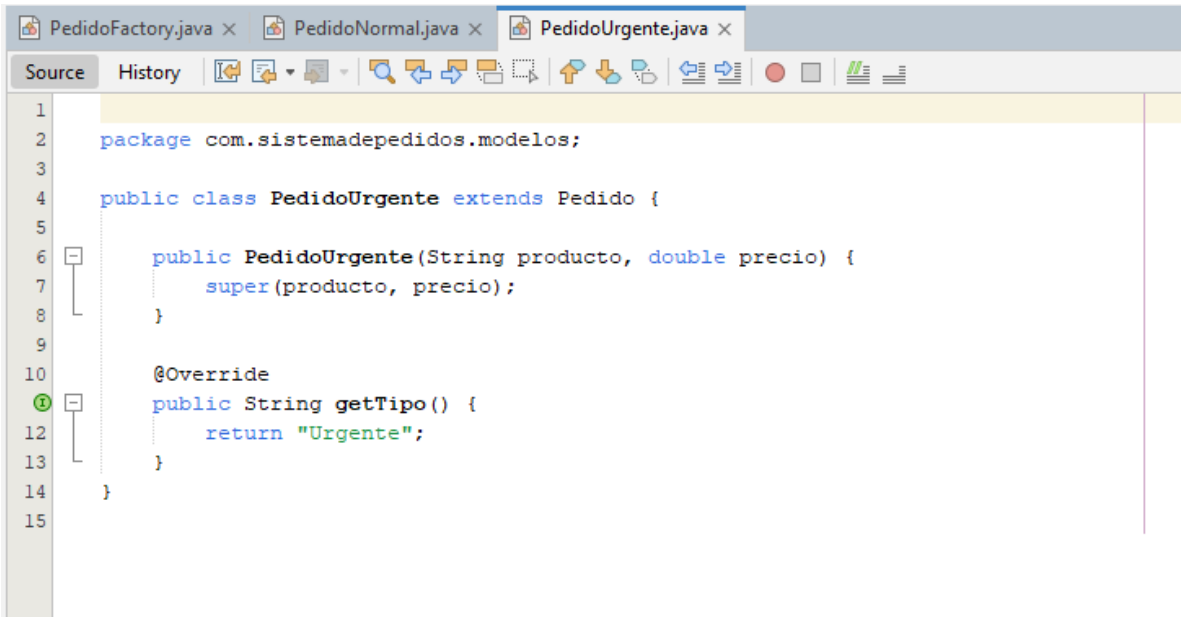
package com.sistemadepedidos.modelos;

public class PedidoNormal extends Pedido {

    public PedidoNormal(String producto, double precio) {
        super(producto, precio);
    }

    @Override
    public String getTipo() {
        return "Normal";
    }
}


```

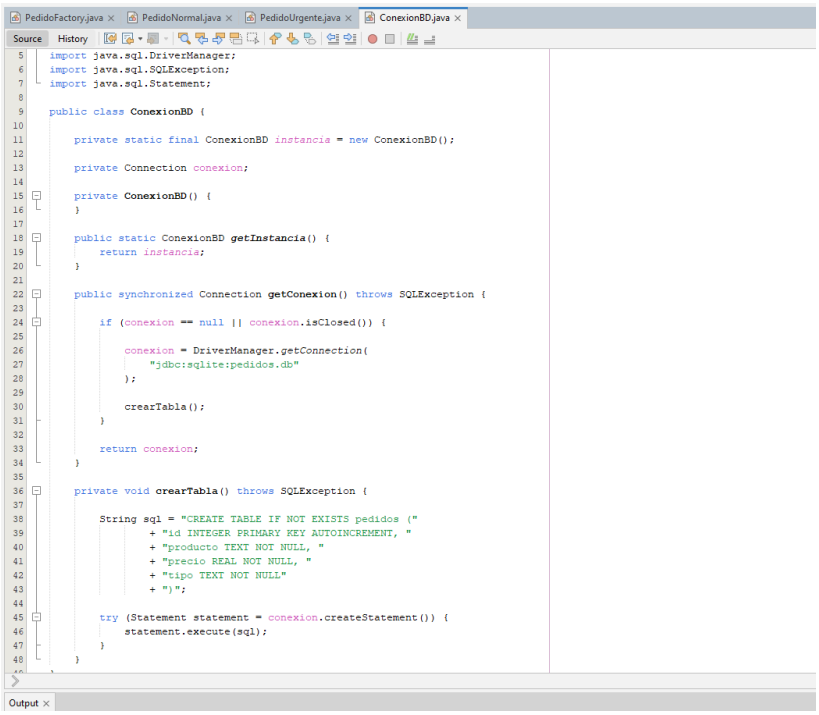


Patrón Singleton

El patrón Singleton se implementa en ConexionBD. La clase mantiene una instancia estática privada y un constructor privado, y proporciona el método getInstancia() para acceder a ella.

El método getConexion() crea la conexión SQLite cuando todavía no existe o cuando se encuentra cerrada, y reutiliza la conexión disponible.

Este patrón centraliza el acceso a la base de datos y evita que distintas partes del programa creen instancias independientes de la clase de conexión.



Patrón Observer

El patrón Observer se implementa mediante las clases Observador, Evento y UsuarioObservador.

Observador define el método actualizar(). Evento mantiene una lista de observadores y permite suscribirlos, cancelar su suscripción y notificarlos. UsuarioObservador implementa la actualización y muestra un mensaje dirigido al usuario registrado.

Cuando ServicioPedidos registra correctamente un pedido, ejecuta evento.notificar(pedido). Como resultado, todos los usuarios suscritos reciben automáticamente la información del nuevo pedido.

```
Pedido registrado correctamente!
ID: 6
Producto: MiniLed Pantalla
Precio: $5200.0
Tipo: Urgente
NOTIFICACION PARA PixelRaam
Se ha registrado un pedido: MiniLed Pantalla
Tipo: Urgente

==============================
```

8. Aplicación de los principios SOLID

Principio de responsabilidad única (SRP)

Cada clase tiene una responsabilidad principal. Pedido representa la información común de un pedido; PedidoFactory crea objetos; ConexionBD administra la conexión; Evento gestiona las suscripciones y notificaciones; RepositorioPedidosSQLite almacena los datos; y ServicioPedidos coordina el proceso de registro.

Esta separación facilita localizar, probar y modificar cada parte del sistema sin afectar innecesariamente a las demás.

Principio abierto/cerrado (OCP)

La jerarquía de pedidos permite incorporar nuevas clases derivadas de Pedido sin cambiar la estructura común de los pedidos. Sin embargo, en la versión actual también debe actualizarse el método crearPedido de PedidoFactory para reconocer un nuevo tipo.

Por ello, el diseño aplica parcialmente este principio y podría fortalecerse utilizando un registro extensible de creadores.

Principio de sustitución de Liskov (LSP)

PedidoNormal y PedidoUrgente pueden utilizarse en cualquier parte donde se espere un Pedido, ya que ambas clases heredan de Pedido y respetan su comportamiento: contienen producto y precio, y proporcionan el tipo mediante el método getTipo().

Esto permite trabajar con los diferentes tipos de pedidos sin modificar la lógica que utiliza la clase base.

Principio de segregación de interfaces (ISP)

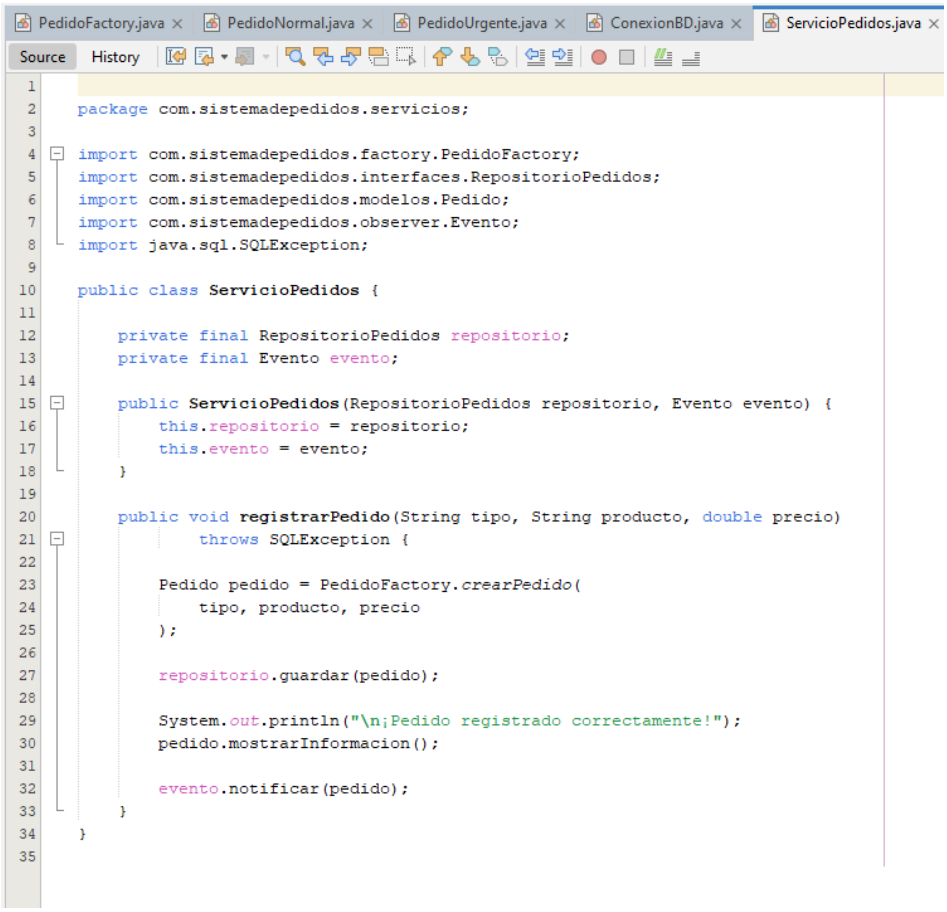
La interfaz RepositorioPedidos contiene una operación específica, guardar(Pedido), en lugar de obligar a sus implementaciones a depender de métodos que no necesitan.

La interfaz Observador también define únicamente la operación necesaria para recibir actualizaciones. De esta manera, las interfaces son pequeñas y específicas para las responsabilidades de cada componente.

Principio de inversión de dependencias (DIP)

ServicioPedidos depende de la abstracción RepositorioPedidos y no directamente de RepositorioPedidosSQLite.

La implementación concreta se entrega desde la clase principal mediante el constructor del servicio. Esto permite sustituir el mecanismo de almacenamiento por otra implementación sin reescribir la lógica central del servicio.



9. Base de datos

La aplicación utiliza SQLite y crea automáticamente la tabla pedidos si todavía no existe.

La tabla contiene los siguientes campos:

id: identificador único del pedido, configurado como clave primaria autoincremental.

producto: nombre del producto solicitado.

precio: costo del producto.

tipo: clasificación del pedido, que puede ser normal o urgente.

La información se guarda en el archivo pedidos.db, generado en el directorio de ejecución del proyecto. Los registros permanecen disponibles después de cerrar y volver a ejecutar la aplicación.

10. Pruebas de funcionamiento

Se realizaron pruebas desde el menú de consola para comprobar el funcionamiento de las principales características del sistema.

Primero se registraron pedidos normales y urgentes. Después se consultaron los pedidos y se comprobó que permanecían almacenados en SQLite. También se registró un usuario observador antes de crear nuevos pedidos y se verificó que recibiera notificaciones automáticas.

Los resultados obtenidos fueron los siguientes:

- Registro de pedido normal: correcto.
- Registro de pedido urgente: correcto.
- Almacenamiento persistente en SQLite: correcto.
- Generación de identificadores automáticos: correcta.
- Consulta de pedidos registrados: correcta.
- Suscripción de usuario y recepción de notificaciones: correctas.

```
===== PEDIDOS REGISTRADOS =====
--------------------------
ID: 1
Producto: Pedido1
Precio: $1800.0
Tipo: Normal
--------------------------
ID: 2
Producto: Pedido2
Precio: $1900.0
Tipo: Urgente
--------------------------
ID: 3
Producto: Laptop Asus Gamer
Precio: $18000.0
Tipo: Normal
--------------------------
ID: 4
Producto: Mouse Gamer Asus
Precio: $1400.0
Tipo: Urgente
--------------------------
ID: 5
Producto: Laptop Asus
Precio: $17000.0
Tipo: Urgente
--------------------------
ID: 6
Producto: MiniLed Pantalla
Precio: $5200.0
Tipo: Urgente

------------------------------
```

```
==============================
     SISTEMA DE PEDIDOS
==============================
1. Crear pedido normal
2. Crear pedido urgente
3. Registrar usuario
4. Consultar pedidos
5. Salir
Selecciona una opcion: 2
Nombre del producto: MiniLed Pantalla
Precio: 5200

❓Pedido registrado correctamente!
ID: 6
Producto: MiniLed Pantalla
Precio: $5200.0
Tipo: Urgente
NOTIFICACION PARA PixelRaam
Se ha registrado un pedido: MiniLed Pantalla
Tipo: Urgente
```

11. Instrucciones de ejecución

Para ejecutar el proyecto en otro equipo se deben seguir los siguientes pasos:

1. Abrir el proyecto SistemaPedidos desde Apache NetBeans.
2. Verificar que el equipo tenga instalado un JDK compatible con el proyecto.
3. Permitir que Maven descargue la dependencia sqlite-jdbc indicada en el archivo pom.xml.
4. Ejecutar la clase principal com.sistemadepedidos.SistemaPedidos.
5. Utilizar las opciones del menú para registrar usuarios, crear pedidos y consultar los registros.

Conclusión

El desarrollo permitió construir una aplicación funcional para registrar y consultar pedidos, almacenar la información de manera persistente y notificar a los usuarios suscritos.

La implementación de los patrones Factory, Singleton y Observer permitió distribuir responsabilidades y organizar la creación de objetos, la conexión a la base de datos y la comunicación entre componentes.

Asimismo, el uso de interfaces, herencia e inyección de dependencias permitió aplicar los principios SOLID en distintas partes del sistema.

Aunque existen oportunidades de mejora, como hacer que la fábrica sea completamente extensible y agregar validaciones de entrada, la versión desarrollada cumple las funciones principales planteadas para el proyecto.